OS VCS Project

Name: S Prithika
SRN: PES2UG24CS415

https://github.com/prithzzz/PES2UG24CS415-pes-vcs

Phase 1:
1A

```
$ ./test_objects
Stored blob with hash: 29dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
Object stored at: .pes/objects/29/dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
```

1B

```
$ find .pes/objects -type f
.pes/objects/29/dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
.pes/objects/91/5b398e3af0ded287e1b03df7656fa7824921ee7c3c839c32f8e220b7923d3e
.pes/objects/21/c4e079678b04e3403b28d0299b5f91fc6af09dad12ccf20e99c6e710020322
```

Q1.1
Objects are sharded into subdirectories to avoid having too many files in a single directory.
A flat directory with millions of files causes slow lookup, creation, and deletion due to filesystem limits.
Sharding keeps directories small, improving performance and scalability.
Q1.2

If we write directly to the final path and the process crashes midway, it can leave a corrupted or partially written object, breaking integrity.

Using temp file + rename ensures:

- File is fully written before becoming visible
- No partial data is exposed

POSIX guarantees that rename() is atomic, meaning:

- It either completely replaces the file or does nothing
- No intermediate (corrupted) state is visible

Q1.3

Git is migrating to SHA-256 because:

- SHA-1 is cryptographically broken (collision attacks exist)
- SHA-256 provides stronger security and long-term reliability

Phase 2:

2A

```
$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
All Phase 2 tests passed.
```

2B

```
$ xxd .pes/objects/9c/0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f708090 | head -20
00000000: 31 30 30 36 34 34 20 52  45 41 44 4d 45 2e 6d 64  100644 README.md
00000010: 00 aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  ................
00000020: aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  ................
00000030: aa 31 30 30 37 35 35 20  62 75 69 6c 64 2e 73 68  .100755 build.sh
00000040: 00 cc cc cc cc cc cc cc  cc cc cc cc cc cc cc cc  ................
00000050: cc cc cc cc cc cc cc cc  cc cc cc cc cc cc cc cc  ................
00000060: cc 34 30 30 30 30 20 73  72 63 00 bb bb bb bb bb  .40000 src......
00000070: bb bb bb bb bb bb bb bb  bb bb bb bb bb bb bb bb  ................
```

Q2.1

Tree entries must be sorted to ensure deterministic hashing.

If entries were stored in filesystem order, two developers with the same files could generate different tree objects (different hashes) due to varying directory listing orders.

Sorting guarantees the same structure → same hash → consistency across systems.

Q2.2

The VCS tracks 100644 vs 100755 to preserve the executable permission of files.

If not tracked, executable files (like scripts) may lose execution permission when checked out, causing them to fail to run or behave incorrectly.

Q2.3

Trees only store entries that point to objects (files or subtrees).

An empty directory has no entries, so it produces no tree object and cannot be referenced.

Thus, empty directories cannot be represented, this is a structural limitation of the object model, not just a design choice.

Phase 3:

3A

```
$ ./pes init
Initialized empty PES repository in .pes/
$ echo "hello" > file1.txt
$ echo "world" > file2.txt
$ ./pes add file1.txt file2.txt
$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
Unstaged changes:
  (nothing to show)
Untracked files:
  (nothing to show)
```

3B

```
$ cat .pes/index
100644 5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03 1776401720 6 file1.txt
100644 e258d248fda94c63753607f7c4494ee0fcbe92f1a76bfdac795c9d84101eb317 1776401740 6 file2.txt
```

Q3.1

The index stores mtime and size to avoid re-hashing file contents, making `pes status` faster by comparing metadata instead of full file data.

Worst case: a file is modified but its size and mtime remain unchanged (e.g., manual timestamp modification), causing the change to go undetected.

Q3.2

Entries must be sorted to ensure deterministic ordering.

If not sorted, the same files could produce different index files across systems, leading to inconsistent behavior and incorrect comparisons.

Q3.3

Atomic write prevents corruption.
If we write directly and crash midway, the index file could be partially written, leaving the repository in a corrupted or unusable state.

Using temp + rename ensures either:

- old index remains intact OR
- new index is fully written

Phase 4:

4A

```
$ ./pes log
commit e57d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
Author: Shanmuga Vel <PES2UG24AM083>
Date:   1776402705
    Add farewell
commit c8b55c9e55bcb9baa90b7db510fe63fa93307b2f617610657d161f0efcc294f1
Author: Shanmuga Vel <PES2UG24AM083>
Date:   1776402645
    Add world
commit 5cd737ea9ff0249d2c888c2b60c2c3cca7f1a5560f43ca4c949db1dbc1d9244f
Author: Shanmuga Vel <PES2UG24AM083>
Date:   1776402585
    Initial commit
```

4B

```
$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/refs/heads/main
.pes/objects/58/91b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03
.pes/objects/e2/58d248fda94c63753607f7c4494ee0fcbe92f1a76bfdac795c9d84101eb317
.pes/objects/5c/d737ea9ff0249d2c888c2b60c2c3cca7f1a5560f43ca4c949db1dbc1d9244f
.pes/objects/c8/b55c9e55bcb9baa90b7db510fe63fa93307b2f617610657d161f0efcc294f1
.pes/objects/e5/7d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
.pes/objects/9c/0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f708090
```

4C

```
$ cat .pes/refs/heads/main
e57d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
$ cat .pes/HEAD
ref: refs/heads/main
```

Q4.1

The ref file stores the latest commit hash. Updating it makes the new commit reachable from HEAD, so it becomes part of history. Until the ref is updated, the commit exists but is not linked to the repository history.

Q4.2

The repository remains consistent. The commit object is written but not referenced, making it an unreachable commit. It is still recoverable if its hash is known. This is safe because no existing data is corrupted.

Q4.3

If writing in-place crashes midway (e.g., only 32/64 hex characters written), the ref file becomes **corrupted and invalid**.
The repository loses the pointer to the latest commit, potentially breaking history access.

Using temp + rename ensures the ref is always either:

- old valid hash OR
- fully written new hash

Phase 5:

Q5.1

To implement pes checkout <branch>:

Update .pes/HEAD to point to the new branch (ref: refs/heads/<branch>)

Read the commit hash from .pes/refs/heads/<branch>

Load the commit → tree → reconstruct files in the working directory

Update the index to match the checked-out tree

The working directory must be rewritten to match the target commit snapshot.

Complexity:

Handling conflicts with uncommitted changes, file deletions, and overwriting files safely makes checkout complex.

Q5.2

Detect dirty working directory using:

For each indexed file:

Compare current file's mtime and size with index

If different → file is modified

Then: Compare the file's blob hash in index with the blob hash in target branch tree

If both differ → conflict

If conflict exists → refuse checkout.

Q5.3

In detached HEAD:

- New commits are created but not linked to any branch
- They become unreachable once you move away

Recovery:

- Use the commit hash directly
- Create a branch pointing to it:

Phase 6:

Q6.1

Algorithm:

1. Start from all branch heads (refs)
2. Traverse commits recursively (follow parent pointers)
3. For each commit:
    - mark commit
    - mark its tree
    - recursively mark all subtrees and blobs
4. Delete all unmarked objects

Use:

- Hash set to track reachable objects efficiently

Estimate:

- 100,000 commits + trees + blobs
  ≈ hundreds of thousands to a few million objects visited

Q6.2

Race condition:

- GC scans objects and marks reachable ones
- Meanwhile, a commit is being created:
  - commit references new tree/blob
  - GC hasn't seen them yet → marks as unreachable
  - GC deletes them

Result: commit points to missing objects (corruption)

Git avoids this by:

- Running GC with locks
- Using safe points
- Keeping recently created objects protected (grace period)